

STRATÉGIES D'AUTHENTIFICATION



Vous souhaitez mettre en place une authentification sur votre site web ...

→ À quoi faut-il penser ?

 Vérification du mot de passe

 Réinitialisation du mot de passe
(dont envoi d'e-mail)

 Gestion des rôles & permissions
(utilisateurs, admins, ...)

 Se connecter avec Google ou Microsoft ?

 Inscription de nouveaux utilisateurs

 Génération et vérification des JWTs
(secrets & algorithmes robustes)

 Gestion de la 2FA (TOTP, Yubikey, SMS, ...)

 Déconnexion et révocation des JWTs

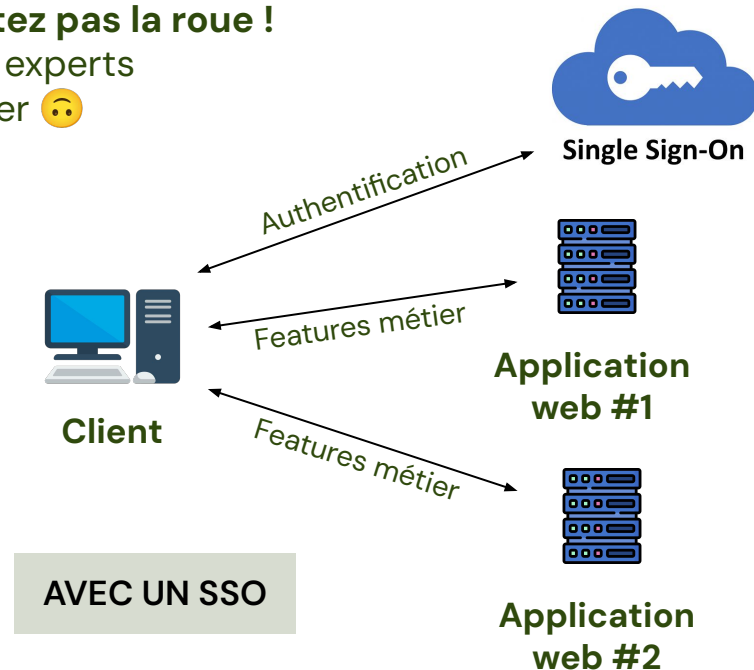
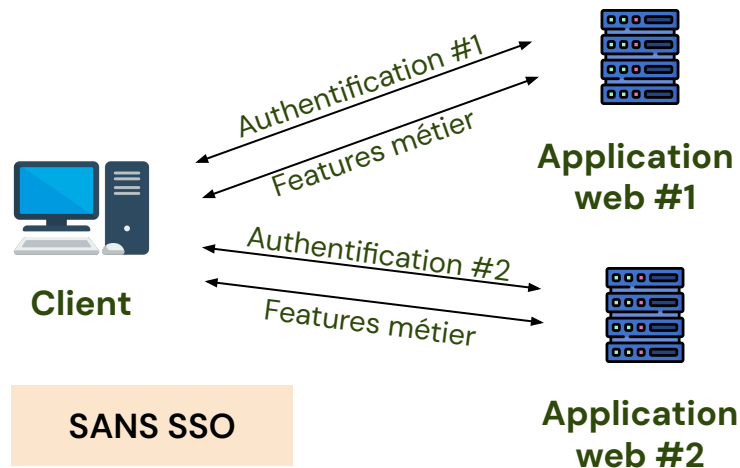
Le Single Sign-On (SSO) à la rescousse !

Objectif : **Retirer de l'application la mécanique d'authentification et d'identité**

→ permet de mutualiser l'authentification sur plusieurs applications !

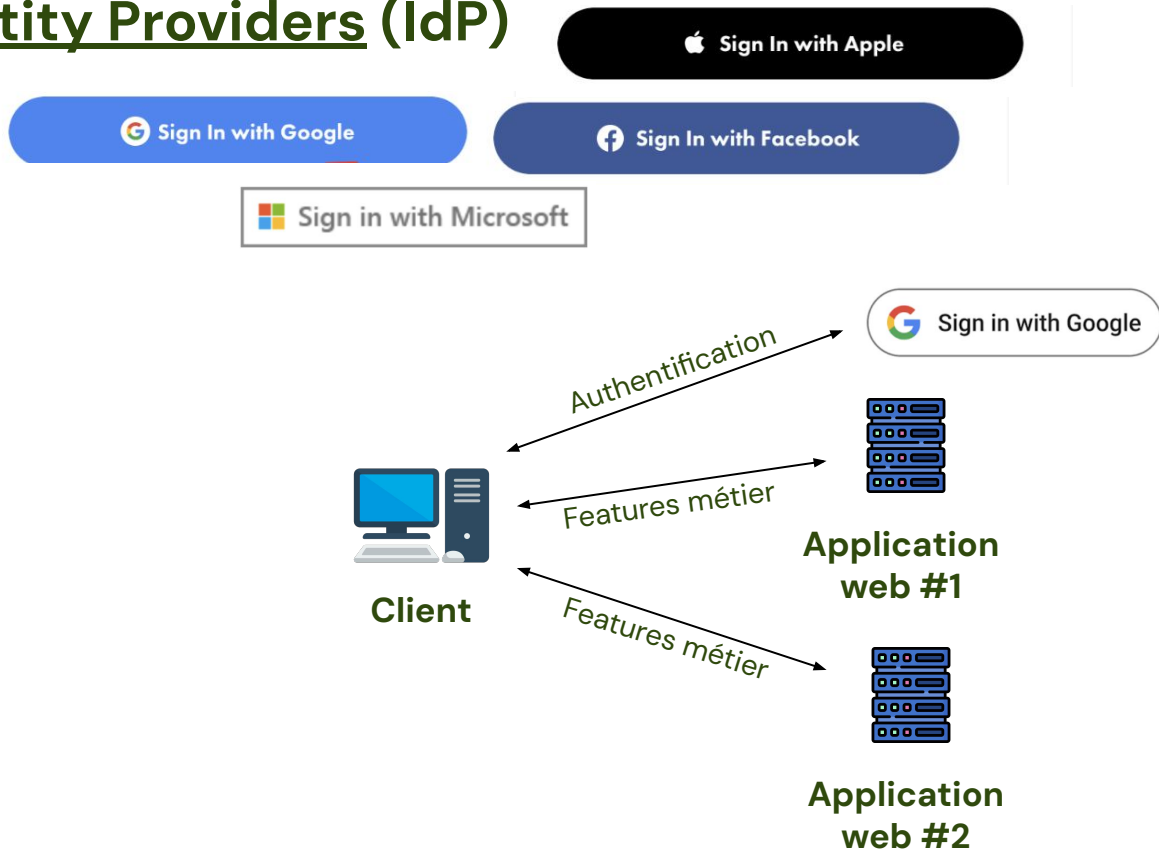
→ **Ce n'est pas votre coeur de métier, ne ré-inventez pas la roue !**

Des projets maintenus depuis plus de 10 ans par des experts
seront toujours mieux que ce vous pourrez développer 🙄



Plusieurs types d'Identity Providers (IdP)

→ Privés, dans le cloud

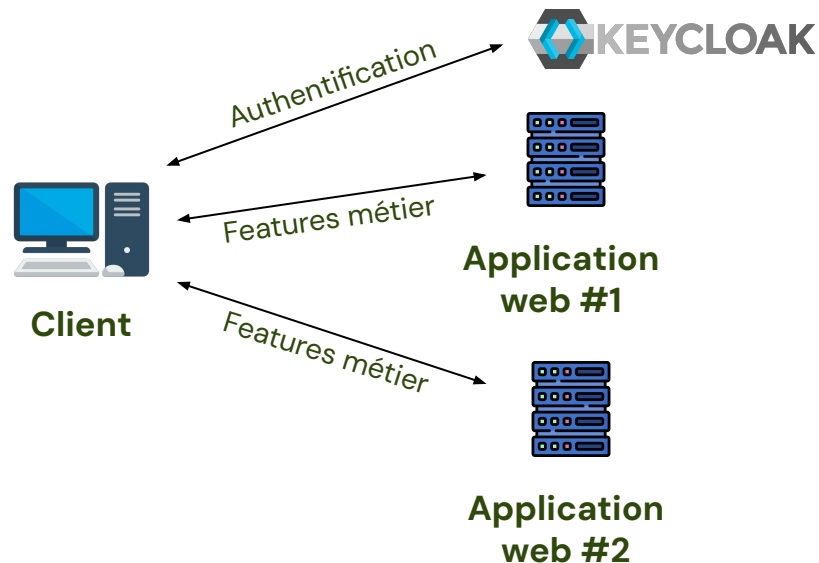


Plusieurs types d'Identity Providers (IdP)

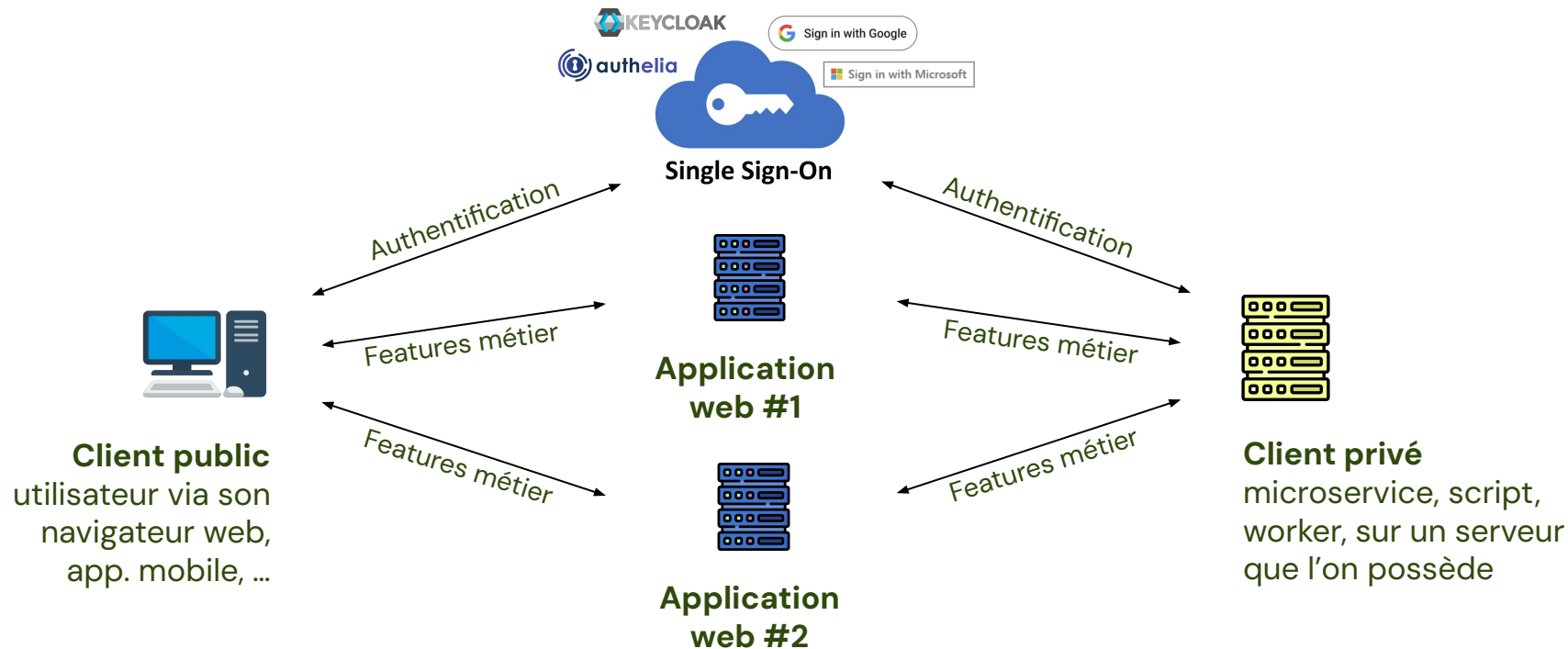
→ Privés, dans le cloud



→ Open-source, auto-hébergés



Plusieurs types de clients



Si plusieurs **clients** échangent avec un même **Identity Provider**,
il faut **se mettre d'accord sur la manière de communiquer !**



OAuth 2

SAML

Format de communication		
Gestion de l'identité des utilisateurs (nom, prénom, e-mail, ...)	Pas nativement → OpenID Connect	
Type de tokens	Access & Refresh tokens	XML documents
Usages & adoption	APIs, apps mobiles	Anciens systèmes
Complexité	Modérée	Élevée

OAuth2 – Direct Flow

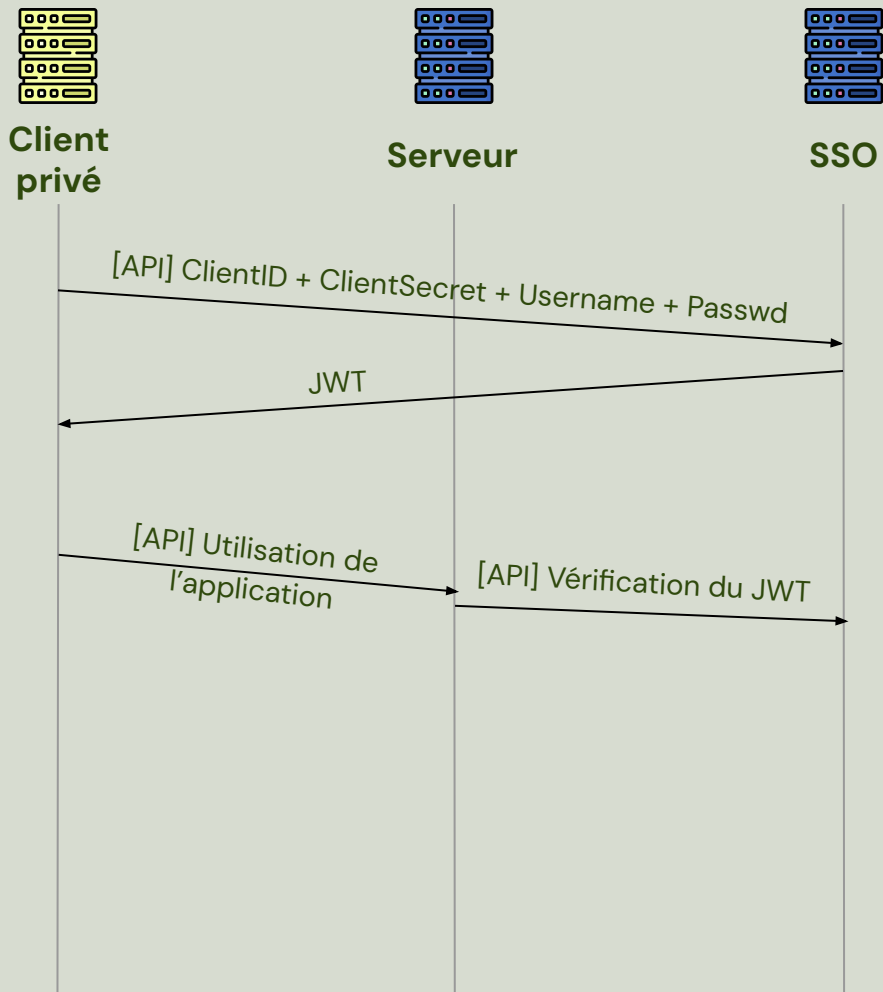
Procédé pour s'authentifier via API
"machine-to-machine" (serveurs, scripts, ...)

ClientID / ClientSecret :

Vérifier quelle application fait la demande
(plusieurs applications peuvent s'authentifier
via le même serveur SSO)

Username / Passwd :

Vérifier quel utilisateur fait la demande



OAuth2 – Implicit Flow

Ancien procédé pour s'authentifier sur une application dans un navigateur web

ClientID :

Idem, mais sans ClientSecret (inutile de le stocker dans le front-end car le JS est "public")

Username / Passwd / (MFA) :

Vérifier quel utilisateur fait la demande

→ Le front ne fait pas une requête directe à l'API du serveur SSO pour s'authentifier car l'authentification peut être plus complexe qu'un login + mot de passe (TOTP, Yubikey, changement de mot de passe à la première connexion, ...)

✗ Ne pas utiliser, car JWT interceptable dans l'URL
(ex: logs apache/nginx qui enregistrent les URL)



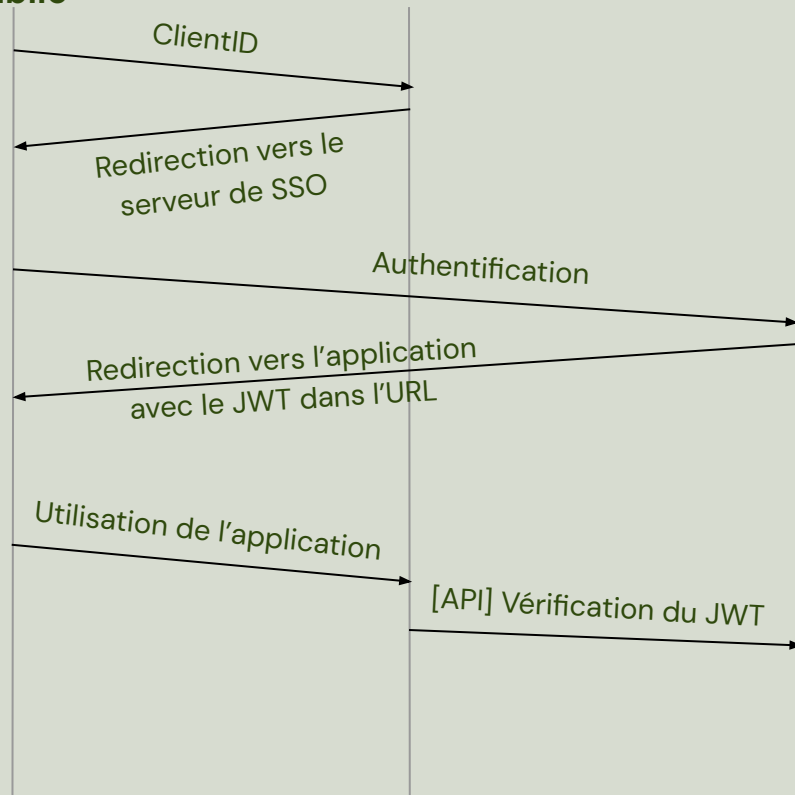
Client
public



Serveur



SSO

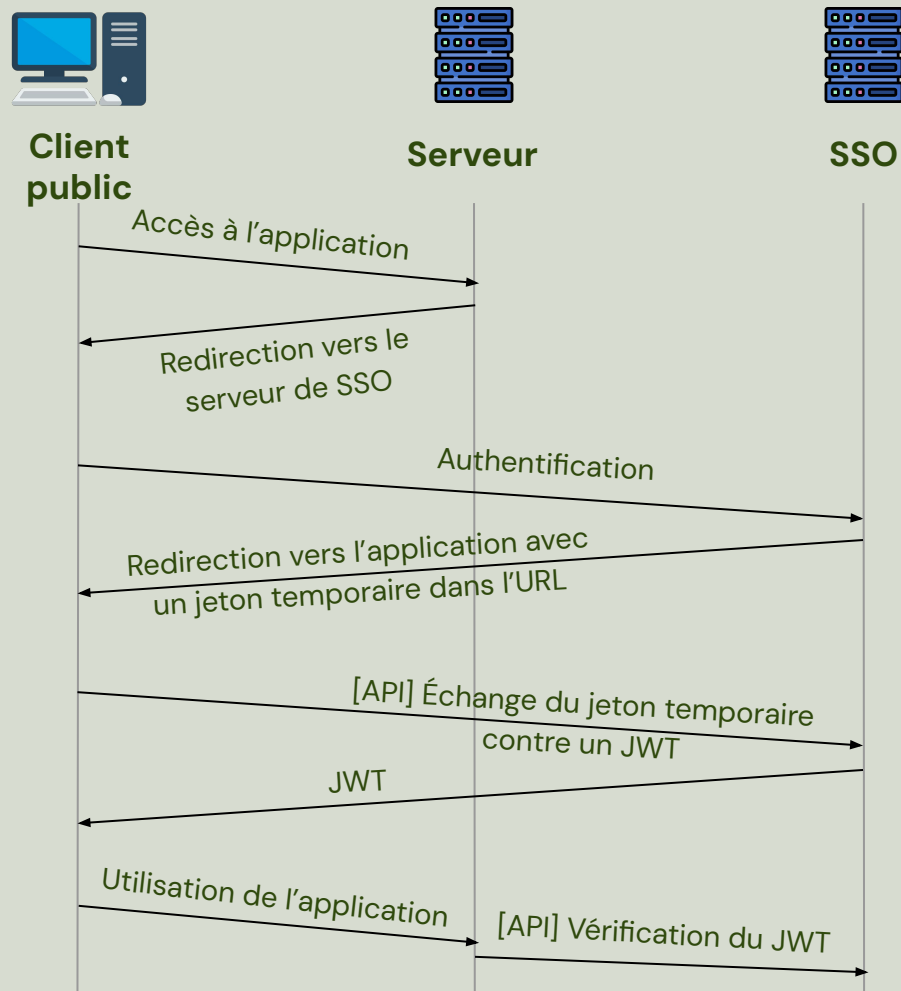


OAuth2 – Standard Flow

Nouveau procédé pour s'authentifier sur une application dans un navigateur web

Très similaire à l'implicit flow, mais :

- ✓ Le JWT n'est plus échangé dans l'URL, mais uniquement un jeton temporaire valable 1 fois qui permet d'obtenir le JWT via l'API du SSO



→ Si notre application ne génère plus les JWTs, comment peut-on vérifier leur signature ?



Signature d'un JWT

Remember!

eyJ0eXAiOiSU0EifQ.eyJpYXQiO0dHJ1ZQp9.KMUFsIDTnFmQV30

Header

Payload

Signature

Chiffrement	Génération de la signature	Vérification de la signature
Symétrique alg : HMAC (HS256, HS512, ...) Adapté si la même application génère et vérifie la signature	 Secret commun	
Asymétrique alg : RSA (RS256, RS512, ...), ECDSA Adapté si une application génère la signature et une ou plusieurs autres applications vérifie la signature (SSO)	 Clé privée	 Clé publique

JSON Web Key Set (JWKS)

Chiffrement	Génération de la signature	Vérification de la signature
Asymétrique alg : RSA (RS256, RS512, ...), ECDSA	 Clé privée	 Clé publique

Conservée secrète en interne par
Keycloak pour signer les JWTs

Disponible publiquement pour vérifier
la signature des JWTs émis par Keycloak

`/realms/{realm-name}/protocol/openid-connect/certs`

→ il s'agit du **JSON Web Key Set (JWKS)**

JSON Web Key Set (JWKS)

/realms/{realm-name}/protocol/openid-connect/certs

▼ keys:

▼ 0:

```
kid: "zYYJTHRuD1DMZlh98vjpgdvJSOPa1qfZgubuUYPa3dQ"
kty: "RSA"
alg: "RS256"
use: "sig"
n: "6ciQ0dtFkcB3wm48s7ypZk-0R8c350V-00qpk33qdskbZoTo_crEsNNde8Y9pip11t
z3UKnqpbA0kWSgoC0qNDuYkiY0j0r6UBsYEUJXfMzPKDPd0MtJ_73B0m_jbftuGw32V"
e: "AQAB"
x5c:
  ▼ 0: "MIICsTCCA ZkCBgGQX27LmzANBgqhkiG9w0BAQsFADAcMRowGAYDVQQDDBF3b3Jrci:
crEsNNde8Y9pip11bqAPoNRiA8oL8UHa2IQVIaw7Esx7dLHuF6WiXi8cF39MEvSrFq:
o4pQxvXKWiEhShh0VJBfx0mULd5NmMRFMFfvQDp1T0E5EIneqe3VzPQIDAQABMA0G
vdrwOnmRd0GbT0sqirum+uSNR4GM6Jv8dZYkvnDbhYBQHkQlc2eQmS83kZbBNzWE+b;
x5t: "KsT4jvJf86BytpaDAS0pM_uwMgA"
x5t#S256: "eCraDyggxZLRhnumV2QfRVBjEQbXpFKuVHAjqEdNqgEA"
```

▼ 1:

```
kid: "kHt037ioVyprMTNZpo788uB2nfBYaCmJsYyoFtxTLw"
kty: "RSA"
alg: "RSA-OAEP"
use: "enc"
n: "r8YZNZRqrFcUgvoPz-umfIJE5hc0Bjx05ZhIWHdp9t0nD09t13sm2_tvSzFP9iNln/
GURLrik4nI2yt2NPPG9XzujMM5Su6xtr5TeNvnJYoQ0rW0cBW2p-1zh3yAwdgPWyeGc
e: "AQAB"
x5c:
  ▼ 0: "MIICsTCCA ZkCBgGQX27NzjANBgqhkiG9w0BAQsFADAcMRowGAYDVQQDDBF3b3Jrci:
```



HEADER:

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "MDc3Q...TI3NEVCQzM5Nw"
}
```

Keycloak peut utiliser plusieurs clés

→ on retrouve l'**ID de la clé utilisée ("kid")**
dans l'en-tête du JWT

OpenID Connect (OIDC)

“Surcouche” ajoutée à OAuth 2 pour **fournir l’identité des utilisateurs** (nom, prénom, e-mail, ...) aux différents clients qui utilisent le SSO

- en plus d’émettre un **Access Token (JWT, contenant les rôles et permissions)** l’Identity Provider émet un **ID Token** (également sous la forme d’un **JWT**) contenant **l’identité de l’utilisateur**



ID TOKEN

The user has been
authenticated ✓

Specs: OpenID Connect



ACCESS TOKEN

The app has been
authorized ✓

Specs: OAuth

PAYLOAD:

```
{
  "iss": "http://my-domain.auth0.com",
  "sub": "auth0|123456",
  "aud": "1234abcdef",
  "exp": 1311281970,
  "iat": 1311280970,
  "name": "Jane Doe",
  "given_name": "Jane",
  "family_name": "Doe"
}
```

Holds
identification
information

SIGNATURE:

RSA SHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),

PAYLOAD:

```
{
  "iss": "https://tenant.auth0.com/",
  "sub": "MCHiB...OsMFrzyb@clients",
  "aud": "https://glossary.com",
  "iat": 1627566690,
  "exp": 1627653090,
  "azp": "MCHiB1T...qQ30OsMFrzyb",
  "scope": "create:term update:term",
  "gt": "authorization_code",
}
```

Holds
authorization
information

SIGNATURE:

RSA SHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),

Refresh Tokens

Génération d'un JWT à expiration courte (quelques minutes) et d'un Refresh Token (séquence aléatoire) permettant d'obtenir un nouveau JWT dans un délai imparti sans nouvelle authentification.

→ Supporté par OAuth 2 et OIDC

- ✓ Réduit fortement les risques de ré-utilisation des JWT en cas de fuite
- ✓ "Déconnexion naturelle" en cas d'inactivité (non-renouvellement à temps du JWT)



Client



Serveur

Authentification
login = kate, pass = *****

JWT valable 10 minutes
+ Refresh Token à usage unique

⋮ Requêtes classiques

Renouvellement du JWT
Refresh Token à usage unique

Nouveau JWT valable 10 minutes
+ Refresh Token à usage unique

**Génération
d'un JWT et d'un
Refresh Token**

**Génération
d'un nouveau JWT et
d'un nouveau
Refresh Token**

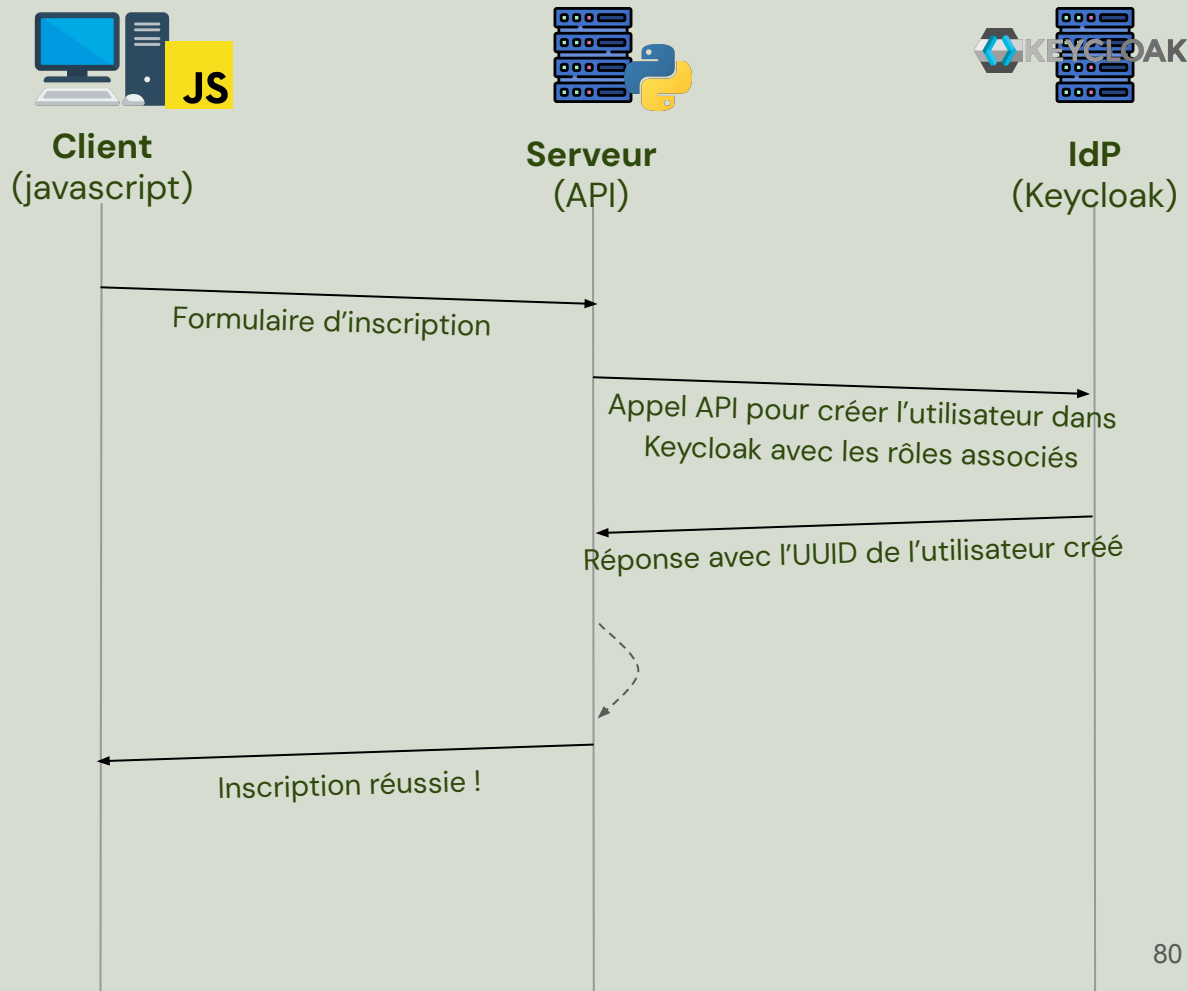
En résumé : Inscription

Résumé concret pour :

- **SPA** (Single-Page Application)
- **Standard Flow OAuth 2**
- Système de **JWKs**



Conseil : utilisez le même UUID
entre l'utilisateur dans Keycloak
et dans votre base de données
pour faire facilement le lien



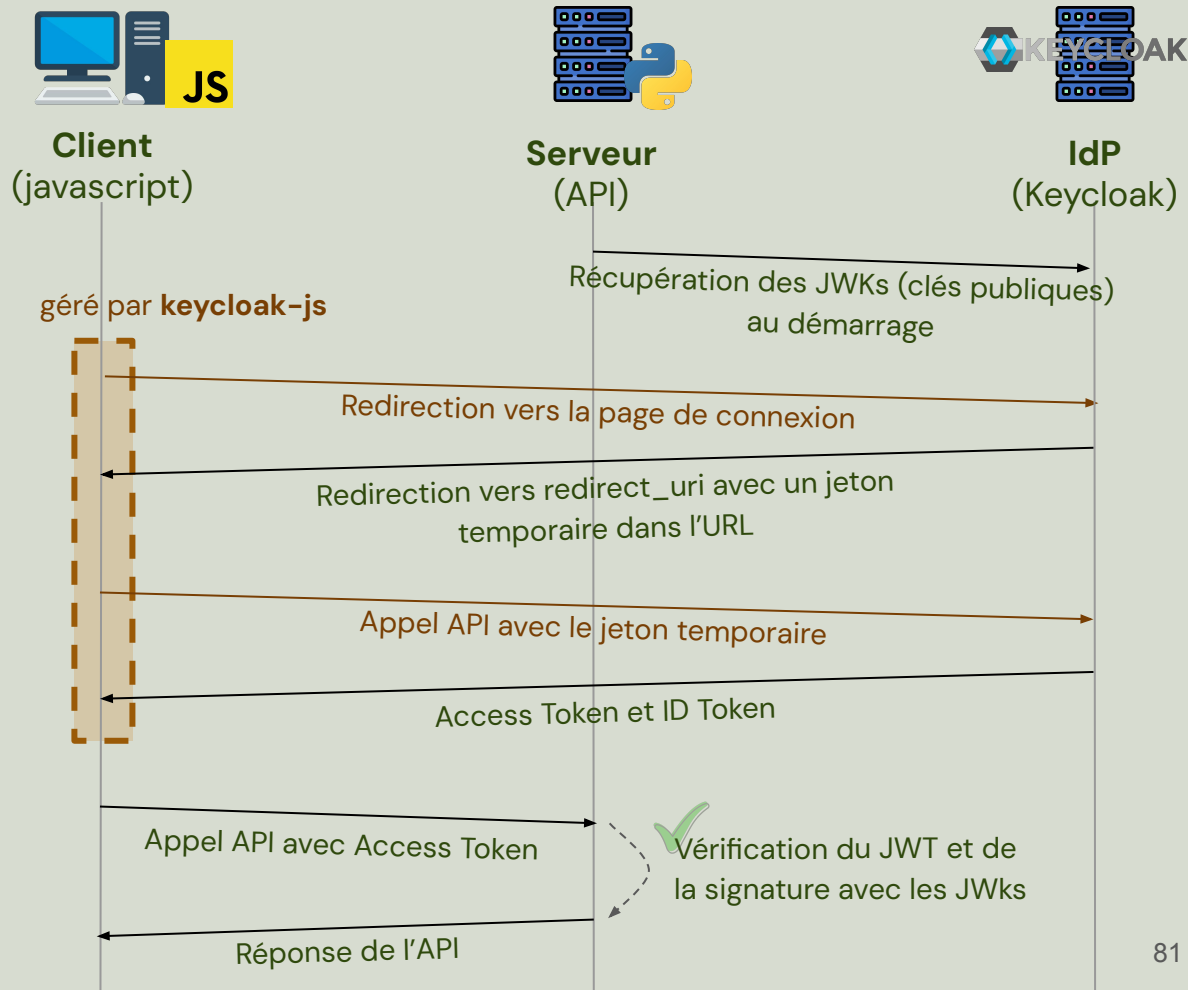
En résumé : Connexion

Résumé concret pour :

- SPA (Single-Page Application)
- **Standard Flow OAuth 2**
- Système de **JWKs**

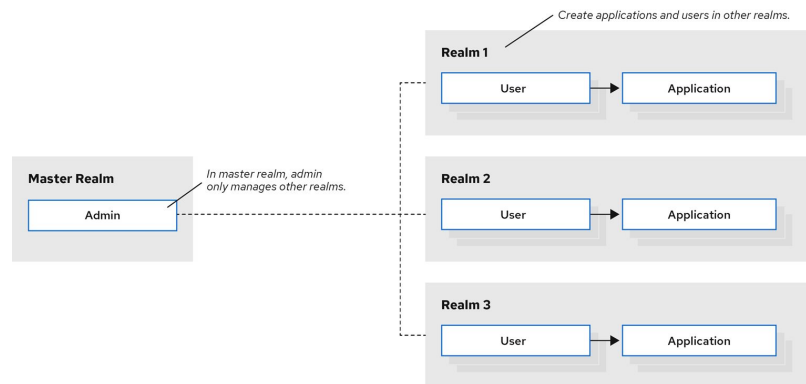
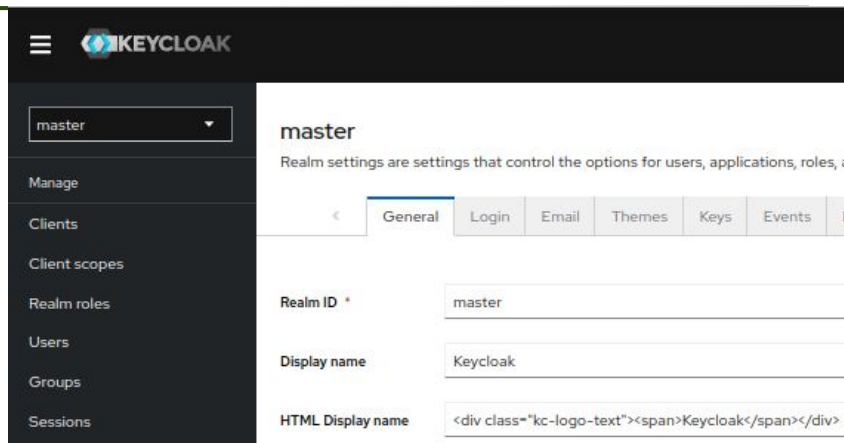
keycloak-js 

26.2.4 • Public • Published 3 months ago



Concepts de KEYCLOAK

- **Realm** : espace d'isolation (config, utilisateurs, clés)
- **Client** : appli/service protégé (front, API, CLI)
- **Users** : comptes humains
- **Service Accounts** : comptes techniques
- **Roles** : realm roles & client roles
- **Groups** : regroupent des users
- **Mappers** : lient attributs ↔ rôles/claims
- **Policies** : règles d'accès
- **Account Console** : gestion autonome du compte de chaque utilisateur
- **Admin Console** : configuration centrale (UI)



RHSSO_44_0919